

GemeindeSim: a bilingual civic town on Apertus 1.5

Hack Apertus 2026 · Track 2B (own project) · Team [hatif03](#) · 5–15 October 2026 · code, logs and the full research record: [track_2b/research](#), [track_2b/docs/research](#)

1. Summary

GemeindeSim reads a Swiss municipal vote in German and French and spawns a small town of residents who talk, move, change mood and form a stance on a pixel-art map; a closing report says who is affected and why, never how to vote. We also used the town as a **test bench for Apertus 1.5** (8B and 70B): about 10,800 logged model calls in 25 experiments (with follow-ups) and 48 instrumented simulations. Apertus is an excellent *speaker* (112/112 residents' lines in the right language; it abstains when a premise is missing; it never gave voting advice in 30 tests). It is an unreliable *decider*: asked for a stance, every resident said **yes** (15/15, also with no persona); on 54 real Swiss federal votes the simulated electorate was 34–37 points too favourable and the 8B followed an official recommendation 98 % of the time; both sizes defined *Steuerfuss* wrongly and failed a one-step percentage. The design follows: **the model voices; the application owns the stance, the arithmetic and the sources**. Result: valid citations 0 → 100 %, invented outcomes in reports 12/30 → 0/30, a town that starts with real disagreement, and residents whose words match their stance ($\approx 0.5 \rightarrow \approx 0.9$). On a real 48-page Federal Council booklet the pipeline answered 12/14 questions correctly and abstained on all unanswerable ones.

2. Architecture and target deployment

```
policy text (DE/FR/EN) → chunk + BM25 (closed corpus, per-chunk language, ballot question pinned)
  → parse_policy (sectors, impacts, ideological valence) → generate_npcs (persona, impact, pro/con arguments in own language)
  → stance = valence×leaning + 0.5×impact - 0.6×computed household cost (code) → N rounds (standard or swarm graph):
    retrieve → reflect → plan → 1-3 typed events (chat/move/protest/mood/price) → code: numeral + citation check, DE B→ss
    → opinion dynamics in Python (bounded confidence; drift only for residents who conversed)
  → Socket.IO → Next.js + Phaser (map, stance poll, citations) → report (stance poll from code, no outcome, no advice)
```

Deployability. Two Docker services (FastAPI/LangGraph backend, Next.js frontend); the corpus is a read-only volume; the only outbound call is the OpenAI-compatible endpoint `LLM_BASE_URL`. **Primary (c):** Swiss-hosted endpoint (the hackathon gateway now). **(a) on-premise / (b) air-gapped:** point `LLM_BASE_URL` at a local vLLM serving `swiss-ai/Apertus-v1.5-70B` or `-8B`. **Status, honestly:** configured and the clean-checkout stack was started and probed; the model behaviours below were measured on the hosted gateway only. No GPU was available, so `research/probe_endpoint.py` (the same table against any endpoint) is ready for a GPU host and was validated against the hosted 8B. The 8B is about 4× faster per simulation than the 70B and fits modest hardware.

3. Use of Apertus

Call	Model	Mode	Notes (measured)
policy parse, persona, pro/con arguments, resident turns, report	70B (8B supported)	json_object, thinking off	an <i>instruction-placeholder</i> example in the resident's language: 49/49 first-attempt valid, 0 % parroting (a filled example is parroted: 8B 53 %)
DE↔FR chat	same	separate translation call	one language per completion; fluent (read)
retrieval, numbers, stance, dynamics	none	Python	no tool calls: parallel calls fail (11/11); thinking + JSON silently skips reasoning

Concurrency is an adaptive limiter (slow start from 4, halve on 429/5xx, back off when each answer gets slower per token): the hackathon gateway answers 429 above 4–5 in flight, CSCS showed no limit to 96; a 429 is retried on the *same* model. A reminder paragraph as the *last* paragraph of the resident prompt binds speech to the code-owned stance; undecided residents get one named pro and one named con argument in new wording.

4. Data

Synthetic bilingual Linden Steuerfuss / school-credit text (`data/`, plus a balanced variant with the referendum committee's arguments). No personal data. For evaluation only: Swissvotes (Univ. Bern), 54 federal votes 2021–2026 with results; one real Federal Council booklet (28 Sept 2025, 48 pages) downloaded by script, not vendored.

5. Evaluation (hosted gateway, 5 October 2026; every call is logged)

5.1 Limits of Apertus 1.5 in an agent loop

Behaviour	Result
Parallel tool calls	fail in 11/11 trials per model, also with <code>tool_choice="required"</code> ; the 70B writes the second call as text
Thinking vs JSON mode	with <code>json_object</code> no reasoning span (16/16); the 70B answers a trap puzzle wrongly in 15 tokens; thinking alone: 6/8 (70B), 8/8 (8B)
<code>temperature 0</code>	5 identical prompts → 3 (70B) and 5 (8B) distinct outputs
Rate limit / speed	429 above 4 in flight; 70B ≈ 40 tok/s per stream (156 at 4), 8B ≈ 158 (555 at 4)
Swiss facts, 15 questions	70B 12/15, 8B 9/15; both define <i>Steuerfuss</i> wrongly; 4,000 × 6 % → 280 (70B), 360 / 72 (8B), correct 240
Language, 112 events	100 % in the resident's language; the 70B writes ß at 2.6 per 1,000 characters (Swiss German has none: filtered in code)
Long context	needle recall 60/60 up to ≈ 105 k tokens (70B ≈ 36 s per call there); single easy fact only

5.2 Does the population behave like a population?

Test	Result
Stance asked directly (15 residents)	15/15 yes under full persona, role-only, no persona; 14/15 with a balanced booklet; impact judged "benefit" for 14/15
54 real Swiss federal votes, ballot title only	simulated yes-share 83 % (DE prompt) / 65 % (FR) vs real 46.5 %; Spearman with real yes-share 0.25–0.38 (70B personas), ≈ 0 (8B); left-right self-placement removes the signal; the Federal Council position alone: 0.63, direction accuracy 0.75
Authority cue	shown the recommendation, the 8B follows it on 98 % of votes (70B 83 %); German prompts are 19 points more favourable than French
Left to itself at the end (E13)	the model agreed with the code stance for 40 % of residents and turned 3 of 4 opponents into "yes" → the stance stays in code

Conclusion: a what-if explainer, not a vote predictor. Compare conditions (with/without an official recommendation, German vs French), not absolute shares.

5.3 Defects found beyond the model, and the fix (identical seeds)

Defect	Before	After
Cited source ids that exist	0 / 159	100 % (16 runs)
Ballot question in the resident's prompt	13–30 %	100 %
Reports asserting "the measure passed"	12 / 30	0 / 30
Reports mixing German and English	5 / 5 runs	0 / 16 runs
Chats opening with a self-introduction (70B)	27 %	0 %
Polarisation of a town where nobody talks (15 rounds)	x 0.51 → 0.69	0 (drift needs a conversation)
Numeral gate, 12 edge cases	8/12	10/12
Influence log reaching the UI	never (dropped by LangGraph)	49–77 outcomes per group
Residents' words match their stance	0.46–0.55 (70B), 0.50–0.52 (8B)	0.88–0.91 (70B), 0.85–0.87 (8B)
Undecided residents sounding undecided ("mixed")	0.47 (70B)	0.74 (70B), 8B 0.53 → 0.68
Retrieval on a real booklet (recall@4, 14 questions)	10/14 (old retriever)	11/14; QA 12/14 hand-graded
<code>make run</code> on a clean checkout	failed (no Linux native packages in either lockfile)	builds; concurrency 4; alt-port configurable

5.4 Real booklet and spread across seeds

Real booklet. On the 48-page Federal Council booklet of 28 Sept 2025 (83 k characters, 209 chunks, two measures) the retriever puts the gold evidence into the four passages for 11 of 14 questions (old retriever 10/14; 12/14 with six passages). With those passages the 70B answered 12/14 correctly (hand-graded; an automatic regex undercounted), abstained on 3/3 unanswerable questions and abstained wrongly only where the evidence was not retrieved: *retrieval, not the model, is the bottleneck* (lexical, exact-term). The French edition could not be tested.

Spread across seeds and conditions. Personas and the stance prior are drawn per seed and $T=0$ is not reproducible, so one run is not a result: across five 70B seeds the final share *for* ranges 0.00–0.80 (8B: 0.00–0.40; its towns are systematically more negative), and the poll did not change in 4 of 5 runs within three rounds. The harness also compares conditions (E19, 8B, three paired seeds): adding the referendum committee's counter-arguments lowers the mean stance in all three seeds (–0.28 at the end); a yes or no council recommendation does not move it consistently, because the stance never reads it.

 stance poll start and end per seed

Stance poll (for / undecided / against) at the start and end of five 70B runs of the same town.

5.5 A second endpoint

The same scripts ran on the CSCS inference API (Swiss National Supercomputing Centre) and, the same day, on the hackathon gateway as a control. The gateway answers 429 above 4–5 requests in flight; CSCS answered 2 496 requests up to 96 in flight without one, and the 5-resident, 3-round simulation took 61 s instead of 126 s on the 70B. Every v2 fix replicates (valid citations 1.00, no self-introductions, no asserted outcome, speech matches the code stance 0.85–0.88) and so do the Swiss-knowledge and real-vote results (Spearman within 0.01). Unchanged: the yes-bias (all four CSCS models, thinking included), one tool call, non-reproducible $T=0$ on real prompts. Strict `json_schema` degenerates on the CSCS 8B (11/24 valid). Both endpoints are hosted; no local run was measured. The 1:1 chat is now grounded like a round (cited passages, number gate, declines unknown facts 5/5, no vote advice 0/5).

5.6 Engineering checks (7 October)

Python is not the bottleneck: process CPU was 2.2–5.0 % of the wall time (70B 60.5 s, 8B 26.1 s for 5 residents × 3 rounds); the wall time is the longest chain of dependent calls at ≈ 60 tokens/s. All calls are `async`; the 1:1 chat streams sentence by sentence only after the number gate and citation check (first text 1.5 s, whole answer 2.4 s). Every call is token-counted from the endpoint's `usage` and shown in a “Run metrics” panel (25 residents × 2 rounds: 143 calls, 168 k tokens in, 51 k out, mean latency 6.6 s, p95 14.4 s; context use at most 1.1 % of the 262 k window). **Retrieval was compared with the usual baselines** on the 14 booklet questions: our BM25 recall@4 0.79, dense multilingual-e5-large 1.00, hybrid BM25 + e5 (RRF) 1.00 (n = 14, one document: indicative); the hybrid is an optional switch (`EMBEDDING_MODEL`), off by default because it needs a 2.2 GB local model. The same build runs on both endpoints (model ids are mapped; `docs/ENDPOINTS.md`): 25 residents × 2 rounds took 89–97 s on CSCS and 654 s on the busy gateway.

6. Limitations

- **Not a vote predictor** (54 real votes; weak signal after bias correction). Stance weights are modelling assumptions, not calibrated; the stance prior *produces* disagreement and is not evidence that real residents would split that way.
- Small n: 3–5 simulations per group, 15 residents in population tests, 15 held-out votes; ranges, no significance tests. LLM classifiers (79–91 % mutual agreement) score the speech tests.
- One endpoint, one day; latency drifts several-fold. Sovereign parity not measured (no GPU). German/French register: native-speaker review pack prepared (`docs/review/`); corrections from the native-speaker review are applied before submission (status in `docs/SUBMISSION-CHECKLIST.md`).
- Open: `price_pressure` indicator is always 0; `invoke_llm_think` is unused; the 8B never emits `move`; the Linden sample is synthetic and short.

7. Reproducibility

`cp track_2b/.env.example track_2b/.env` (set `LLM_API_KEY`), then `make run` → UI `localhost:3000` (second port `UI_ALT_PORT`, default 8080). Tests: `cd track_2b/src/backend && uv run pytest --deselect tests/test_e2e.py` (offline). Every experiment is a script in `research/` with raw call logs in `research/results/`; `docs/research/LAB-NOTEBOOK.md` records every decision, including failed attempts. Python 3.12, Docker.

8. Next steps

Run the probe on a local vLLM; apply the native-speaker corrections; hand-built, statistics-based personas; calibrate the stance prior on municipal results; vote-specific dashboard indicators.

License and references

CC-BY-4.0 (all HackApertus projects are open-sourced); code Apache-2.0. Park et al. 2023 (arXiv:2304.03442); Park et al. 2024 (2411.10109); Peralta et al. 2022 (2201.01322); Chuang et al. 2024 (2311.09618); Barmettler 2026 (2606.00048); von der Heyde et al. 2024 (2409.09045); Hu & Collier 2024 (2402.10811); Atil et al. 2024 (2408.04667); Gao et al. 2023 (2305.14627); Swissvotes, University of Bern; Bundeskanzlei, *Erläuterungen des Bundesrates*, 28.09.2025; Apertus technical report (2509.14233) and Apertus 1.5 documentation; Apertus Charter.